# Concordance: reading Swiss law in three languages at once

**Track:** Track 2B, Own Project **Event:** Online **Author:** Ali Aoun Mehdi (Ali Aoun Mehdi) **Demo video:** to be added **Code:** https://github.com/Ali-Aoun5/concordance-hackapertus **Dataset:** https://huggingface.co/datasets/AliAounMehdi/concordance-trilingual-divergence **Code commit:** `ea766f8`

## 1. Summary

Swiss federal law is written three times. The German, French and Italian versions are equally binding, and there is no original to fall back on. If the German says an authority *may* do something and the French says it *must*, the law says two different things depending on the language you read. Catching that is careful work, done mostly by people, before a text is published.

Concordance is a second pair of eyes for that work. It lines the three versions up sentence by sentence, checks them with transparent rules, asks Apertus 8B to read all three, and has Apertus 70B confirm or reject every candidate. A finding shows the exact words in each language, a severity, the model's reasoning and, where possible, a corrected wording. A reviewer confirms or dismisses it in a small offline interface. It does not interpret the law and it does not decide which version is right. It says "these three disagree here, please look".

I tested it on 136 cases built from three real federal acts. The cascade's confirmed findings reach **precision 0.84 and recall 0.69**. Rules alone find more (recall 0.84) but wrongly flag 69% of clean articles; Apertus 70B alone is almost never wrong (1% false alarms) but finds only 28%. Combining them cuts the rules' false alarms by 81% while keeping most of their recall, at about 69% of the cost of running the 70B model on every article. Everything runs on open Apertus weights, and the demo runs with the network switched off.

| Criterion | Where to look |
|---|---|
| Purposeful use of AI | Section 3 and 5: Apertus is the semantic judge that removes 81% of the rules' false alarms; four modes are compared head to head |
| Technical rigour | Section 5: ground-truth test set, per-type results, two operating points, failures reported; automated tests |
| Value, cost, scalability | Section 5: about 1,900 tokens per article per pass; cascade costs 69% of 70B-on-everything |
| Sovereign deployability | Section 3 and 7: replay with `--network none` ; open weights only; local-server path documented |
| Implementation feasibility | Section 7: one command, Docker only, tested on a fresh clone |

## 2. Architecture

1. **Ingest (offline, once).** Acts are fetched from Fedlex as Akoma Ntoso XML and the three languages are aligned on the language-independent article id. Articles that exist in only some languages would be recorded as structural differences. In the three acts used there were none.
2. **Sentence alignment.** Co-drafted law keeps the same paragraph and sentence structure across languages, so corresponding sentences are paired by position. If the structure does not match, the tool falls back to comparing the whole article.
3. **Rule engine (no model).** Compares numbers, units, dates and cross-references over the article, and modal force (may, must, must not), negation, quantifiers and legal terms sentence by sentence. It handles Swiss number formats and idioms that look like negations but are not (*sans délai*, *ne … que*).
4. **Apertus 8B triage.** Reads all three versions and proposes divergences as strict JSON with verbatim quotes.
5. **Apertus 70B verification.** Only on articles that have candidates (77% of cases). It sees the corresponding sentences and a question specific to the type of difference, and answers confirmed, refuted or uncertain. It may add problems the earlier stages missed.

6. **Guards.** Every quoted span is checked against the source text. Rule findings for numbers, units, dates and cross-references cannot be silently deleted by one model refutation; they are kept as "needs a human decision".
7. **Review interface.** One offline HTML file: three columns side by side, highlighted differences, severity, the verifier's reasoning, confirm / dismiss / ask-an-expert, CSV and JSON export.

A disk cache of model answers sits between the pipeline and the models. That is what lets anyone replay the exact reported numbers without a key or a network connection.

## 3. Use of Apertus

**Models:** `swiss-ai/Apertus-v1.5-8B` and `swiss-ai/Apertus-v1.5-70B`, through the CSCS OpenAI-compatible endpoint. No other model was used. Temperature 0. Prompts are in `src/concordance/prompts.py`.

**Why a model is needed at all.** The rules can see words but not meaning. They cannot tell that French "ne peuvent ... que" means "may only", or that a different word for the same duty is only style. On real law they flagged 69% of clean articles. The language model is the part that reads meaning, and the measured effect is in Section 5.

**Triage prompt (8B).** A system prompt describing a legal-linguistic reviewer, two short worked examples, a list of divergence types and a strict JSON format that demands verbatim quotes.

**Verification prompt (70B).** The three texts, the numbered candidates with their origin, the corresponding sentences and one question per type, for example for modality: "compare only the legal force of the verbs; 'dürfen nur' and 'ne peuvent ... que' mean the same, but 'kann' against 'doit' is a real difference".

**Why Apertus.** The task needs strong German, French and Italian legal reading, and unpublished drafts cannot be sent to a foreign cloud. Apertus is open, multilingual and built in Switzerland. The 8B/70B pair also makes the cost-aware cascade possible.

**Deployment.** Development used the CSCS endpoint (Switzerland). The tool accepts any OpenAI-compatible URL, so an organisation can serve both models on its own GPUs, with a separate URL for the 70B. I documented this path in `docs/architecture.md` but did not run it on local hardware.

## 4. Data

- **Real corpus.** Three consolidated federal acts from Fedlex, retrieved on 2026-10-14, in German, French and Italian: the Federal Act on Data Protection (SR 235.1, 77 articles), the Federal Act on Administrative Procedure (SR 172.021, 88) and the Insurance Contract Act (SR 221.229.1, 98). The evaluation uses the first 60 articles of each act (180 articles). Official legislative texts are not subject to copyright under Art. 5 of the Swiss Copyright Act; source and date are recorded per article.
- **Test set.** From these articles I built 136 cases. 68 have one language version deliberately changed in one place (a number, unit, date, modal verb, negation, quantifier, cross-reference or legal term; the exact change is logged), and 68 are untouched controls. The counts per type are number 2, unit 4, date 7, reference 12, negation 12, modality 12, quantifier 7, terminology 12.
- **Synthetic demo law.** A short invented act with five planted differences, used for the video. It is labelled synthetic everywhere and is not Swiss law.
- No personal data and no human subjects. The test set and model answers are published on Hugging Face (link above) under CC-BY-4.0.

## 5. Evaluation

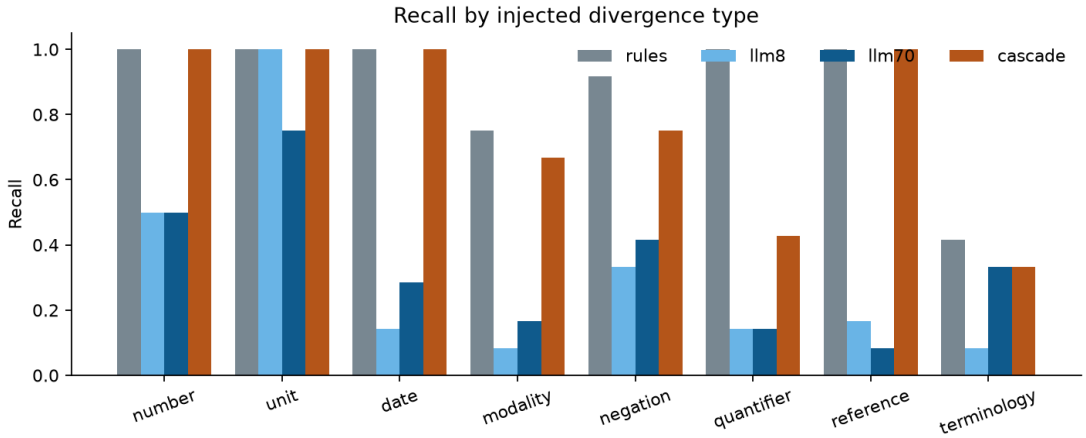
**What is measured.** A case counts as "flagged" if the tool reports any divergence. *Strict* counts only findings the 70B confirmed (the confirmed list); *all findings* also counts those left as "needs a human decision". Cost is in 8B-equivalent tokens: 70B tokens are weighted 70/8, a proxy for compute, not a price.

| Setup | Precision | Recall | F1 | False alarms on clean | Cost |
|---|---|---|---|---|---|
| Rules only | 0.55 | 0.84 | 0.66 | 0.69 | 0 |
| Apertus 8B | 0.83 | 0.22 | 0.35 | 0.04 | 257,076 |
| Apertus 70B on every article | 0.95 | 0.28 | 0.43 | 0.01 | 2,253,519 |
| **Cascade, strict** | 0.84 | 0.69 | 0.76 | 0.13 | 1,565,875 |
| Cascade, all findings | 0.70 | 0.72 | 0.71 | 0.31 | 1,565,875 |

How to read it. Rules alone are too noisy to use. The 70B alone is trustworthy but misses most differences, especially in long articles where one verb changes. The cascade keeps most of the rules' recall and most of the 70B's precision. Strict F1 is 0.76 against 0.66 for rules and 0.43 for the 70B. The 77% of cases that reach the 70B explain why the saving is a modest 31%, not a dramatic one.

**Recall by injected type** (all findings counted; some samples are small):

| Type | n | Rules | Apertus 8B | Apertus 70B | Cascade |
|---|---|---|---|---|---|
| number | 2 | 1.00 | 0.50 | 0.50 | 1.00 |
| unit | 4 | 1.00 | 1.00 | 0.75 | 1.00 |
| date | 7 | 1.00 | 0.14 | 0.29 | 1.00 |
| reference | 12 | 1.00 | 0.17 | 0.08 | 1.00 |
| negation | 12 | 0.92 | 0.33 | 0.42 | 0.75 |
| modality | 12 | 0.75 | 0.08 | 0.17 | 0.67 |
| quantifier | 7 | 1.00 | 0.14 | 0.14 | 0.43 |
| terminology | 12 | 0.42 | 0.08 | 0.33 | 0.33 |




**What went wrong, and what I changed.** The first versions did worse than the numbers above. 1. The 70B rejected real problems, so my first fix made the cascade ignore its rejections. On real law that flooded the output with false alarms. The final rule is narrower: only numbers, units, dates, cross-references and "at least against at most" survive a rejection. 2. The rules compared whole articles, so they set a verb in one sentence against a different verb in another. Aligning sentences fixed most of that and revealed swaps that article-level checks hid. 3. Some of my generated test errors were not real errors (a capitalised "Kann" that opens a conditional clause). I corrected the generator and re-ran everything.

**Real-law run.** On the 180 real articles the cascade raised 116 findings in 66 articles. The interface lists 96 of them as needing a human decision. **These are candidates, not errors.** Real federal law is carefully edited, and the false-alarm rate above says many will turn out to be harmless. They have not been checked by a legal-linguistic expert.

**Cost.** One pass costs about 1,900 tokens per article per model. At that rate a 1,000-article act needs roughly 1.9 million tokens for one pass of the 8B model.

## 6. Limitations

- A finding means the versions appear to differ. Whether that is a defect is a legal judgement.
- The error generator and the rule engine share a word list, so the rules' recall on injected errors is optimistic. The language models do not share it.
- Injected errors are cleaner than real ones. Clean controls could hide real divergences, which would count as false alarms.
- Apertus 70B makes mistakes of its own. It once claimed that "au plus" (at most) means "at least". That is why the verifier cannot overrule the most reliable rule findings.
- Where sentences do not line up across languages, the tool falls back to whole-article checks, which causes most of the remaining modality false alarms (9 on clean articles).
- Terminology recall (0.33) and quantifier recall (0.43) are low, and several test samples are small.
- Only three acts were tested. Romansh, tables and annexes are not covered.
- The local-server deployment path is documented but not run.

## 7. Reproducibility

- **One command:** `make run` (Docker only). It builds the image and runs it with `--network none`, replaying the recorded Apertus answers from `data/cache/`. Output: `out/review.html`, `out/summary.md`, `out/eval_table.md`.
- **Without make:** `docker build -t concordance .` then `docker run --rm --network none -v "${PWD}/out:/app/out" concordance`.
- **Checked on a fresh clone** of this repository: same table as in Section 5, no model call, no warning.
- **Determinism:** temperature 0; test generator seed 7; answers cached by a hash of model, messages and settings. Changing a prompt or the corpus invalidates the cache; `make warm` records it again with a key.
- **Hardware:** replay needs only a CPU. Fresh analysis needs the CSCS endpoint or local GPUs.
- **Tests:** `make test` (all passing).

## 8. Next steps

Compare at paragraph and clause level with a better aligner, so the remaining false alarms shrink. Run the whole Classified Compilation and publish a map of where the three versions most often drift. Check amendments, where a difference is most likely to be introduced. Check term consistency across a whole act using the federal terminology database. Add Romansh. Train a small verifier on reviewer decisions.

## License

Code: Apache-2.0 (see `LICENSE`). Report and datasets: Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

## References

- Fedlex open data and SPARQL endpoint: https://fedlex.data.admin.ch
- Akoma Ntoso (OASIS LegalDocML) 3.0.
- Apertus v1.5, swiss-ai: https://huggingface.co/swiss-ai